Lab: Building a Full-Stack Serverless Chat Application

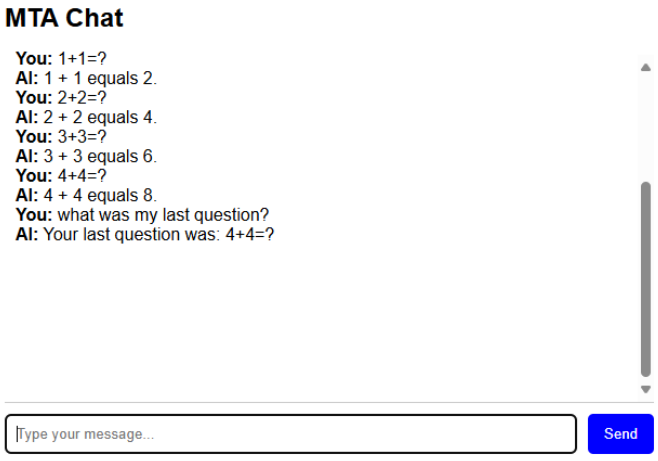
Lab overview and objectives

In this lab, you will build a **Full-Stack Serverless Chat Application** using AWS and OpenAI technologies. The application will incorporate a DynamoDB database to provide the chatbot with memory, allowing it to remember previous questions and responses within the same conversation session.

By the end of this lab, you will have a fully functional web-based chatbot application that supports:

- Real-time interaction with an AI model using OpenAI's Chat Completion API
- A stateless AWS Lambda function enhanced with persistent conversation history stored in DynamoDB
- A Function URL or A REST API interface created with API Gateway
- A simple frontend chat page capable of sending user prompts and displaying AI responses
- Deployment of the frontend as a static website using AWS Amplify

This end-to-end solution demonstrates how to integrate cloud services and AI capabilities into a modern serverless application.



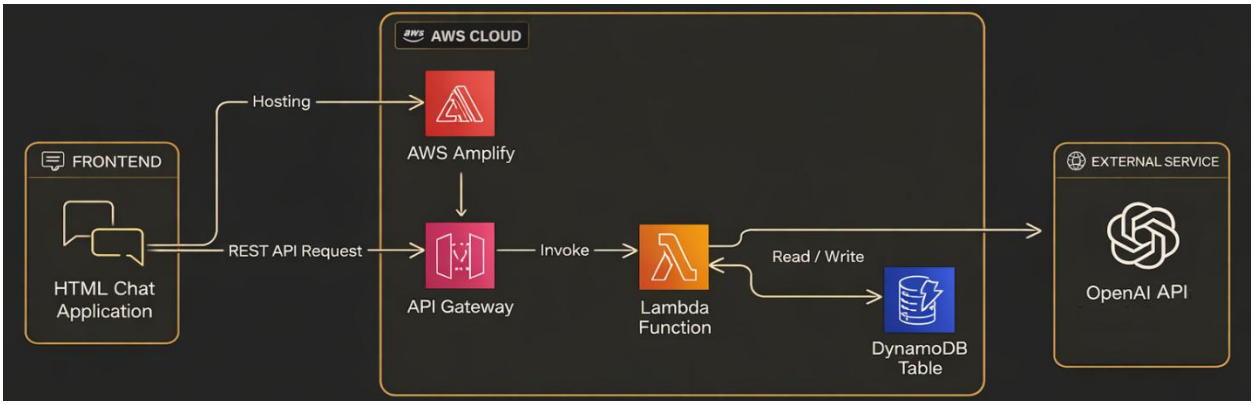
AWS Architecture.

The following diagram illustrates the end-to-end architecture of the serverless chat application you will build in this lab. It shows how the different AWS services and components interact to process user messages, generate AI responses, and maintain conversation history.

This architecture includes:

- **HTML Frontend** hosted on AWS Amplify
- **API Gateway REST API** that exposes an HTTP POST endpoint
- **AWS Lambda function** that processes user prompts, calls the OpenAI API, and reads/writes chat history
- **DynamoDB table** used to store conversation messages and provide memory across interactions

Together, these components form the complete serverless application pipeline you will implement.



Accessing the AWS Management Console

For this lab we will use the **AWS Academy learner** lab and will be using the **AWS Management Console**.

Task 1: Create Python chat using Open AI API

In this task, you will write a simple Python program that communicates with the OpenAI API to generate responses based on user input. This preliminary exercise ensures that you can successfully call the OpenAI API and build a basic chat loop before integrating AI into AWS services later in the lab.

To interact with OpenAI, you must authenticate using an API key.

The API key is a unique secret identifier that authorizes your program to access OpenAI services on behalf of your account. Because this key grants access to your resources, it is essential to keep it private and never expose it in public repositories, client-side code, screenshots, or shared environments.

For this exercise, you will use a hard-coded API key when creating the OpenAI client.

This is acceptable for learning purposes; however, it is not recommended in production.

In real-world applications, API keys should be stored securely using solutions such as:

- Environment variables
- AWS Systems Manager Parameter Store
- AWS Secrets Manager
- Encrypted configuration files

By completing this task, you will verify that your OpenAI setup works correctly and that you can generate responses from a simple Python script before moving forward to AWS integration.

1. Complete the Python program as described below:
    - *Import the OpenAI library:*
        1. *Start by importing the `OpenAI` class from the `openai` module.*
    - *Initialize the OpenAI client:*
        1. *Create an instance of the `OpenAI` client using your API key.*
    - *Set up the conversation history:*
        1. *Initialize an empty list to store the conversation history.*
    - *Create a loop to interact with the user:*
        1. *Use a `while` loop to continuously prompt the user for input until they type 'exit'.*
        2. *Append each user input to the conversation history.*
    - *Generate a response from the AI:*
        1. *Use the `responses.create` method of the OpenAI client to generate a response based on the conversation history.*
    - *Print the AI's response:*
        1. *Print the AI's response to the console.*
        2. *Append the AI's response to the conversation history.*
    - *Use a low-cost model like 'gpt-4.1-nano' model*

    Note about the conversation history: This conversation history is stored only in memory and will reset every time the Lambda function is invoked. In later tasks, we will store chat history in DynamoDB so that the Lambda function can remember previous interactions and pass them to OpenAI.

2. Run the code, verify it's working by asking any question:

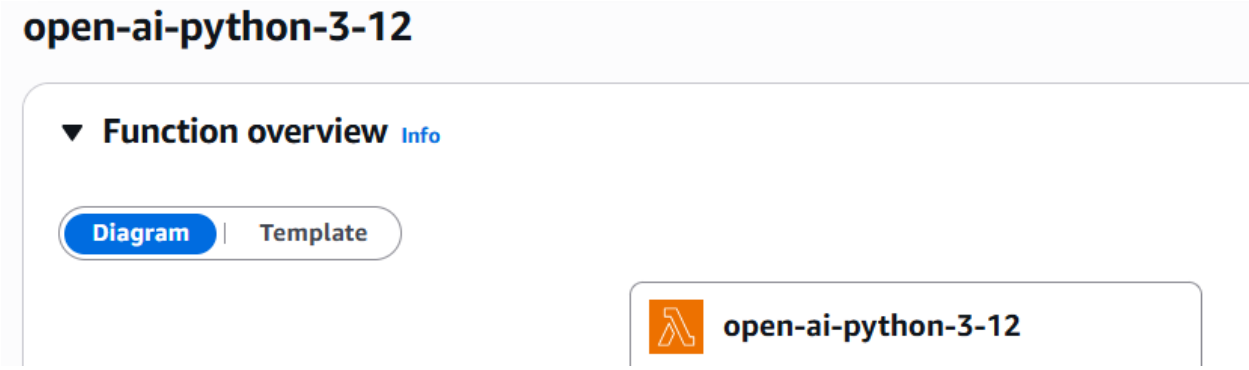
```
Enter your prompt (or 'exit' to quit): what is Open AI API Key?
AI: OpenAI API Key is a unique authentication key provided by OpenAI, a
leading artificial intelligence research lab, that allows developers to
securely access and use their AI models and tools through their API
(Application Programming Interface). The API Key acts as a secure token
that verifies the identity of the user and grants them access to the
resources and services provided by OpenAI. Developers can use this key
to integrate OpenAI's powerful AI capabilities into their own
applications, tools, and services.
Enter your prompt (or 'exit' to quit):
```

Task 2: Create & Deploy a Python AWS Lambda function

In this task, we will write a lambda function in python, based on the code we wrote in task 1.

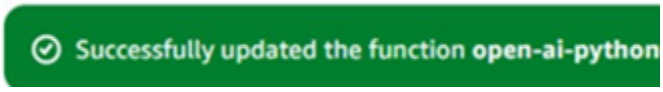
1. Write the next AWS Lambda function in python code:
    - *Import the OpenAI library:*
        1. *Import necessary libraries:*
            1. *Import the `json` module for handling JSON data.*
            2. *Import the OpenAI class from the openai library to interact with the OpenAI API.*
    - *Define the Lambda handler function:*
        1. *Create a function named `lambda_handler` that takes `event` and `context` as parameters.*
    - *Initialize the OpenAI client:*
        1. *Create an instance of the `OpenAI` client using your API key.*
    - *Parse the Request Body:*
        1. *Try to parse the body of the event as JSON. If parsing fails, return a 400 status code with an error message.*
        2. *Extract the user_prompt from the parsed body.*
    - *Validate the user prompt:*
        1. *Check if the `user_prompt` is present. If not, return a 400 status code with an error message.*
    - *Prepare the Conversation History:*
        1. *Initialize an empty list for conversation_history.*
        2. *Append the user_prompt to the conversation_history.*
    - *Generate a response from the AI:*
        1. *Use the `client.responses` method of the OpenAI client to generate a response based on the conversation history.*
    - *Return the AI's response:*
        1. *Return a 200 status code with the AI's response in the body.*
    - *Use a low-cost model like 'gpt-4.1-nano' model*

2. In **AWS Management Console**, In the search box to the right of **Services**, search for and choose **Lambda** to open the AWS Lambda console.
3. Choose **Create a function**.
4. In the **Create function** screen, configure these settings:
    - Choose **Author from scratch**
    - Function name: `open-ai-python-3-12`
    - Runtime: **Python 3.12**
    - Choose **Change default execution role**
    - Execution role: **Use an existing role**
    - Existing role: From the dropdown list, choose **LabRole**
5. Choose **Create function**.
6. The below should appear.



7. Below the **Function overview** pane, choose **Code**, and then choose *lambda_function.py* to display and edit the Lambda function code.
8. In the **Code source** pane, delete the existing code, and paste the code you wrote above.
9. In the **Code source** box, choose **Deploy**.

    Your Lambda function is now fully configured.



Task 3: Verify that the Lambda function works

1. Choose **Test** tab, and update the next JSON in the **Event JSON** panel:

```
{
  "body": "{\"user_prompt\": \"Who are you?\"}"
}
```

2. Press on **Test**. The function will execute, and error will occur:

This error occurs because the AWS Lambda environment does not include the openai package by default. The Lambda runtime can only access libraries that are:

- Built into the runtime, or
- Packaged and deployed together with the function, or
- Provided through a Lambda Layer

Since the openai library and its dependencies are not part of AWS Lambda's default environment, the function cannot import them.

To resolve this issue, you will create an AWS Lambda Layer that contains the OpenAI package.

In the next task, you will package the openai library into a Lambda Layer and attach it to your function so it can import the module successfully.

Task 4: Create AWS Lambda layer for openai

To allow your Lambda function to import and use the openai Python package, you need to create a Lambda Layer. A layer is a separate deployment artifact that contains external libraries and dependencies your function needs. This keeps your function code lightweight and makes the dependencies reusable across multiple Lambda functions.

For this lab, you will use a **pre-created Lambda Layer ZIP file** that already includes the OpenAI library and all required dependencies for the **Python 3.12** runtime. This ensures compatibility with the Lambda runtime used in your function and avoids the need to manually install or package the library yourself.

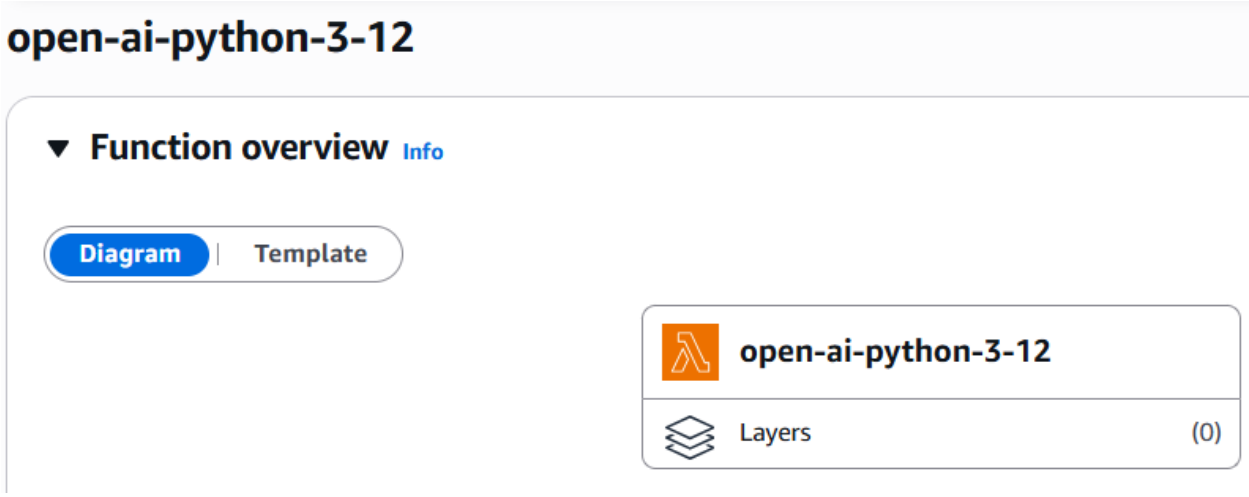
You will upload this ZIP file as a Lambda Layer, configure it for **Python 3.12**, and then attach it to your Lambda function. Once the layer is attached, your Lambda function will be able to successfully import the openai module.

openai-2.24.0-layer-python-12.zip

Task 5: Configure AWS Lambda layers

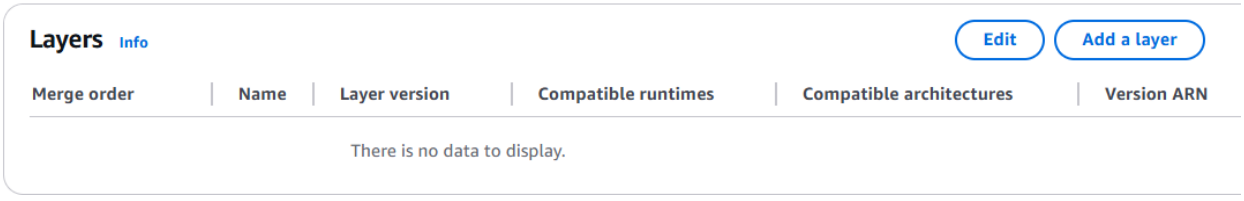
In this task you will solve the issue with missing dependencies using AWS Lambda layers.

1. In the Lambda **Function overview** pane, you will see that there is currently no layer.

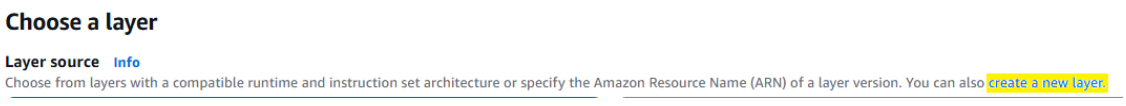


2. In the Lambda Function overview pane, you will see that there are currently no Layers. Press on the Layers link to navigate to the Layers pane:

3. In the **Layers** pane, press on **Add a Layer.**

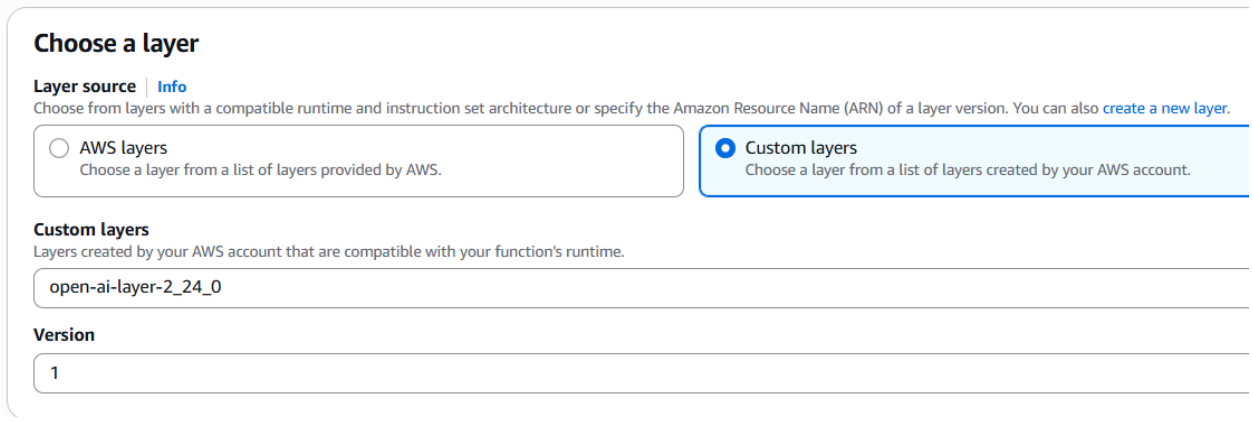


19. In the **Add layer** page, in the **Choose a layer** pane, press right click on the **create a new layer link,** and select Open link in new tab.



20. Navigate to the new tab, and In the **Create layer** window, set the **Name** as **openai-layer**, press on the **Upload** button, and select the **openai-lambda-layer.zip** file. Select the **Compatible runtimes** drop down, select Python 12, and press **Create**. Your layer has now successfully created.

21. Navigate back to the **Create layer** window and refresh the page (F5), in the **Choose a layer** pane, select the **Custom Layers** radio button. Then, open the **Custom Layers** drop down and select **openai-layer**, and in the Version drop down, select **1,** and press on **Add**.



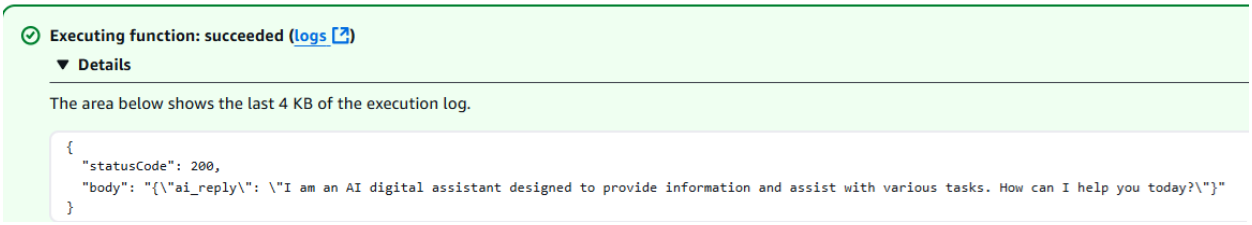
Task 6: Re Verify that the Lambda function works

1. Open the **Test tab** in your Lambda function and run the same test event you used earlier.

   Reminder: update the next JSON in the **Event JSON** panel:

```
{
  "body": "{\"user_prompt\": \"Who are you?\"}"
}
```

2. After the test completes, expand the **Details** section to review the execution results. Examine the following:
   Response – the output returned by your Lambda function
   Summary – duration, memory usage, and billing information
   Logs – including any printed messages and OpenAI API responses
3. If the Lambda Layer was attached correctly, the function should now execute without import errors, and you should see a valid response from the OpenAI API.
   The response will look similar to:
   {
   "statusCode": 200,
   "body": "{\"ai_reply\": \" **I am an AI digital assistant designed to provide information and assist with various tasks. How can I help you today?**\"}"
   }



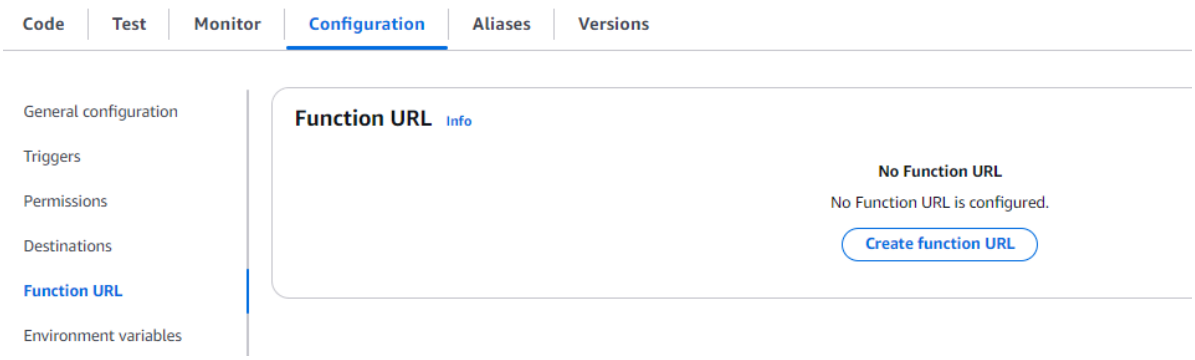
Task 7: Expose the Lambda Function as a Web API

In this task, you will learn how to make your Lambda function accessible from a web browser.
You can choose between two approaches: a simple **Function URL** (basic), or a more advanced and flexible solution using **API Gateway** and a **REST AP**. Based on your choice, continue to Task 7.1 or Task 7.2.

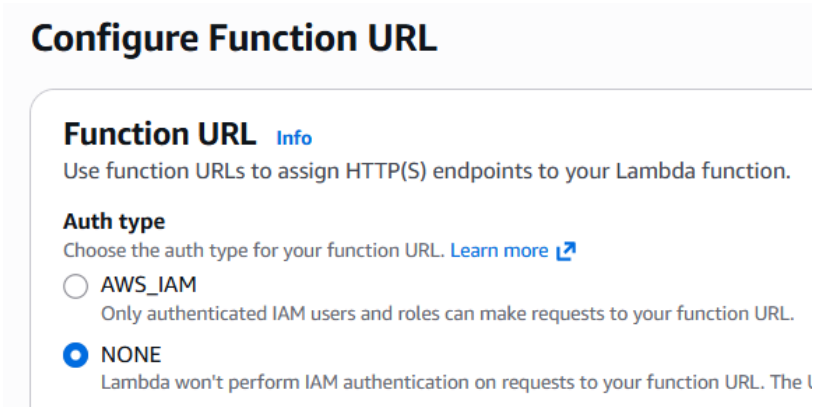
Task 7.1 Expose the Lambda Function using a Function URL (Basic)

In this task, you will expose your Lambda function through **AWS Lambda Function URL**. This will allow external clients such as your HTML chat page (Task 9) or Postman (Task 8) to call the Lambda over HTTP.

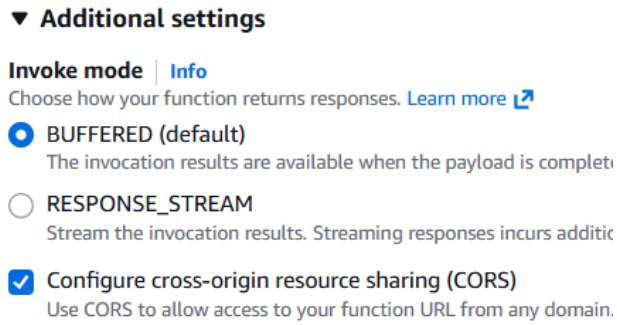
1. Choose **Configuration** tab, **Function URL** (a dedicated HTTP(S) endpoint) option and press on **Create Function URL**.



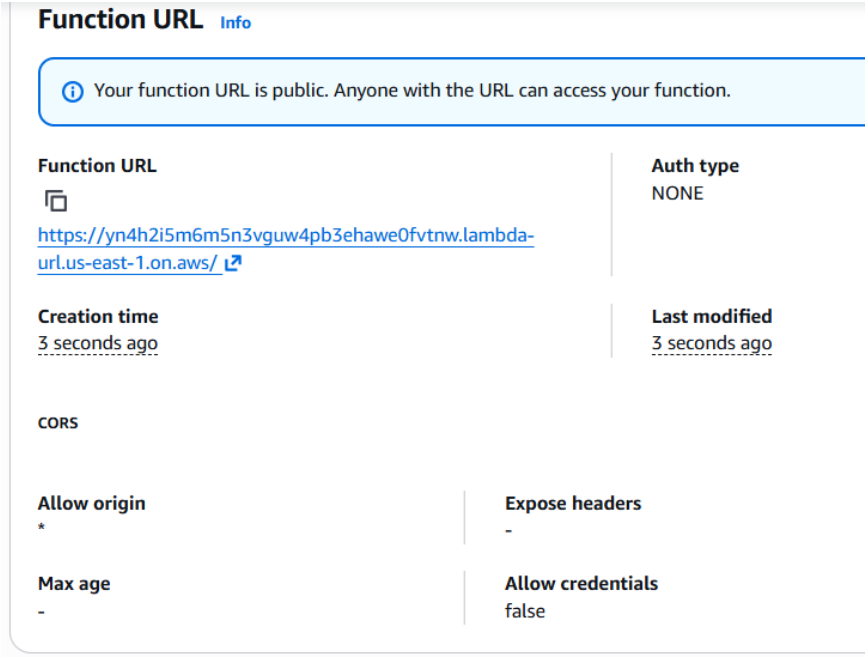
2. In the **Configure Function URL** section, under **Function URL** section select **None**



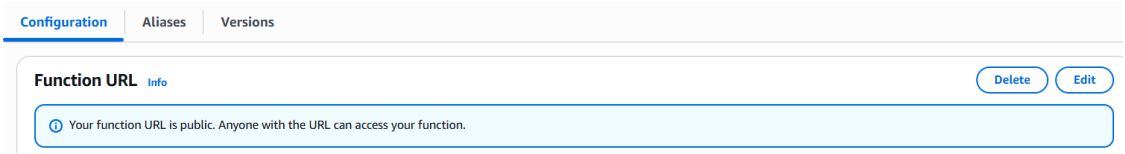
3. Open the **Additional settings** section and select **Configure cross-origin resource sharing (CORS)**



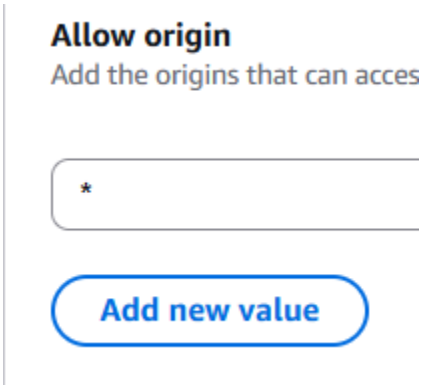
4. Press **Save**. Your Function URL should now be created.



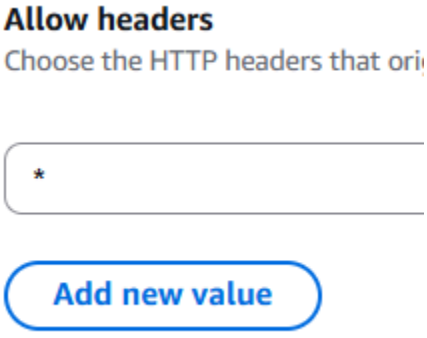
5. Add support to CORS headers in the Lambda function from Function URL Configuration.
    - In the **AWS Lambda function** window, Choose **Configuration** tab, **Function URL** and **Edit**.



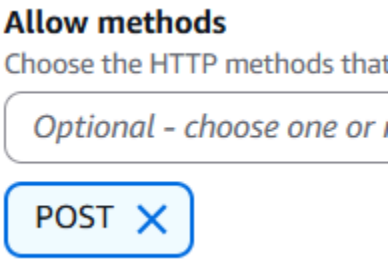
    - In the **Additional settings** section:
        - Make sure that **Allow origin** has the value of *****.



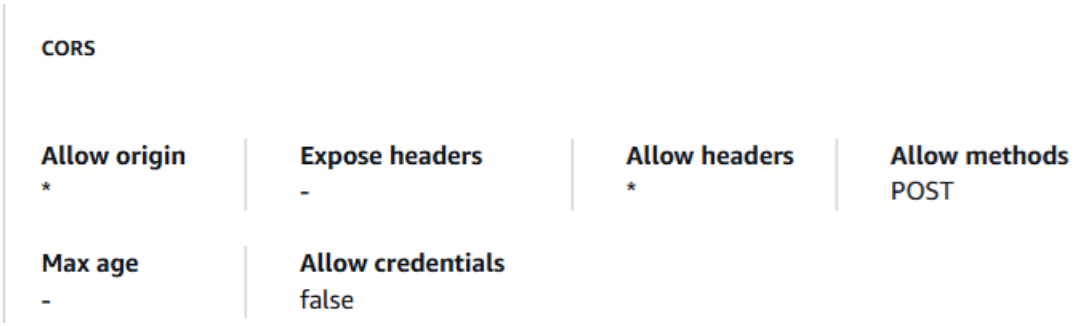
        - Set the Allow headers to * by pressing **Add new value**.



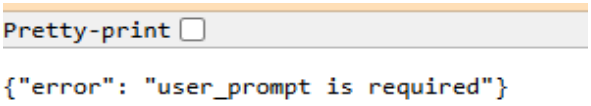
- Open the **Allow methods** drop down, and select **POST**



- Press **Save**. Your Function URL should now be updated.



6. Copy your function URL, and from your preferred web browser, execute it. You should probably get an error of missing user_prompt parameter.



Because our Lambda function supports POST request, we cannot execute it by copying and pasting the URL into a browser's address bar like with a GET request. POST requests require a request body, which cannot be included in a URL.

Task 7.2 Expose the Lambda Function using API Gateway & REST API (Advanced)

In this task, you will expose your Lambda function through an **API Gateway REST API**. This will allow external clients such as your HTML chat page (Task 9) or Postman (Task 8) to call the Lambda over HTTP.

Because API Gateway REST APIs **do not automatically forward request bodies to Lambda**, you must configure a **Mapping Template**.
Additionally, modern web applications require **CORS (Cross-Origin Resource Sharing)**, so proper CORS headers and OPTIONS method configuration are essential.

1. Create a new REST API
   - In the API Gateway console, choose **Create API → REST API (Build)**.
   - Name the API: **open-ai-chat-api**.
   - Create a **Resource**, for example: /chat.
   - Make sure to check the option: **CORS (Cross Origin Resource Sharing)**
2. Add the HTTP Method (POST)
   - Under the new resource, create a **POST** method.
   - Set Integration Type to **Lambda Function**.
   - Select your Lambda function: *openai-python-3-10*

   This POST method will be the entry point for your chatbot.
3. Configure the Request Mapping Template (IMPORTANT)
   API Gateway REST API does *not* automatically pass the raw HTTP request to Lambda.
   You must explicitly map the incoming request body to event["body"].
   Under POST → **Integration Request**:
   - Expand **Mapping Templates**.
   - Set **Content-Type** to: application/json
   - Add the mapping template:

     ```
     {
       "body": $input.json('$')
     }
     ```

   This ensures that when the client sends:
   {"user_prompt": "Hello"}
   your Lambda receives: event["body"]["user_prompt"]
   without needing manual parsing.

4. Configure the Integration Response Mapping Template
   - To ensure your frontend receives clean JSON, set:
     Integration Response → 200 → Mapping Template (application/json):
     $input.json('$.body')
5. Add CORS Support (CRITICAL FOR THE FRONTEND)

   Your HTML chat page (Task 9) cannot call your API unless CORS is configured correctly.
   CORS requires **two things**:

   1. A preflight **OPTIONS** method
   2. Matching CORS headers on the **POST** method

   Important Note:

   If you enabled **CORS** when creating the resource in API Gateway, the **OPTIONS method may already be created automatically**.
   In that case, verify that the generated OPTIONS method includes:

   - Access-Control-Allow-Origin
   - Access-Control-Allow-Headers
   - Access-Control-Allow-Methods

   If the OPTIONS method does not exist (or is missing headers), you must add or correct it manually.

   A. Add or verify the OPTIONS Method

   For the /chat resource:

   1. If OPTIONS already exists → Open it and verify the correct headers.
   2. If it does not exist → Create a new **OPTIONS** method.

   Required Method Response Headers

   Add or Verify:

   - Access-Control-Allow-Origin
   - Access-Control-Allow-Headers
   - Access-Control-Allow-Methods

Required Integration Response Mappings

- header.Access-Control-Allow-Origin → '*'
- header.Access-Control-Allow-Headers → 'Content-Type,Authorization,X-Amz-Date,X-Api-Key,X-Amz-Security-Token'
- header.Access-Control-Allow-Methods → 'DELETE,GET,HEAD,OPTIONS,PATCH,POST,PUT'

B. Add CORS headers to the POST Method as well

Even if OPTIONS handles the preflight, browsers **also require** CORS headers in the actual response of the POST request.

Under POST → **Method Response**, add:

```
Access-Control-Allow-Origin
```

Under POST → **Integration Response**, add:

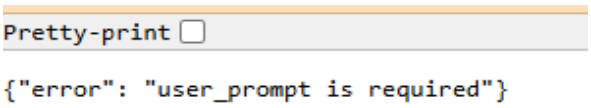
```
Access-Control-Allow-Origin → '*'
```

6. Use the Test tab for the POST method and send JSON:
   {"user_prompt": "Hello"}
   Make sure it works and returns a 200 status code before deploying.
7. Deploy the API

   After confirming the test:

   - Click **Actions** → **Deploy API**
   - Create a stage (e.g., **prod**)

7. Copy the Invoke URL and from your preferred web browser, execute it. You should probably get an error of missing user_prompt parameter.



Because our Lambda function supports POST request, we cannot execute it by copying and pasting the URL into a browser's address bar like with a GET request. POST requests require a request body, which cannot be included in a URL.

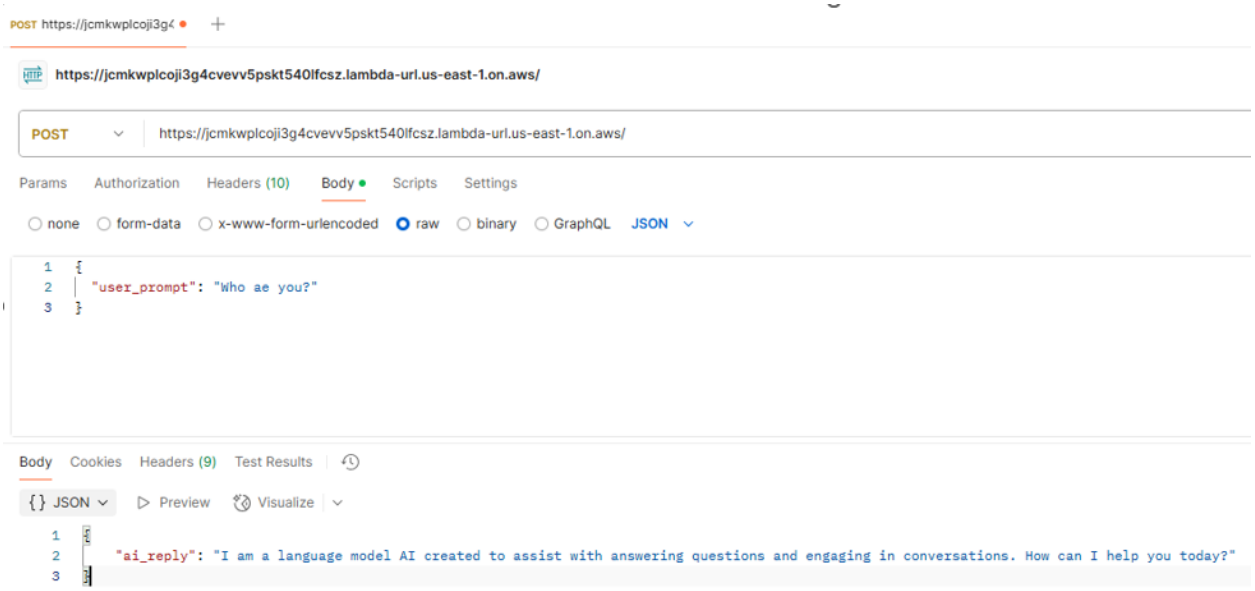
Task 8: Execute the Lambda function from postman

Postman is a popular API testing tool that allows you to send HTTP requests and inspect the responses without writing any code. It is commonly used by developers to test REST APIs, debug issues, and verify backend functionality. In this task, you will use Postman to call your API Gateway endpoint and confirm that it successfully triggers your Lambda function.

1. Open **postman.com** and select to **Send an API request**.

2. Set the request type to POST.
3. Enter the URL of your deployed Lambda function endpoint.
4. Set the headers:
   Key: Content-Type
   Value: application/json
5. Set the body of the request to raw JSON and include the following content:

```
{
  "user_prompt": "Who are you?"
}
```



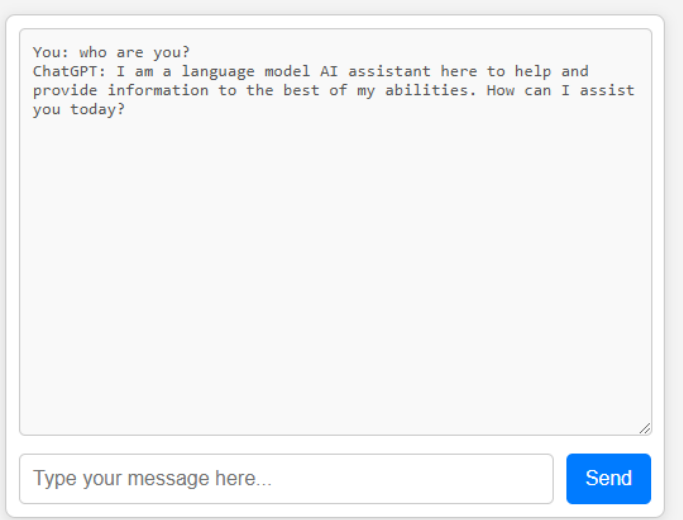
Task 9: Execute the Lambda function from an HTML page.

In this task, you will create a simple HTML page that communicates with your API Gateway REST API and displays chatbot responses in a basic chat interface. This page will allow you to send a prompt to your Lambda function using the **POST** method and render the AI's reply inside the browser.

1. Create a basic HTML page that will execute your lambda function. HTML page will implement a basic chat window. Make sure to execute POST method when calling your Lambda function. Make sure to use your API Gateway REST API URL.
2. You can use GenAI to generate HTML. Sample prompt below (Note: Make sure to replace the ***<REST_URL>*** with your API URL):

    create a basic chat HTML.
    1. It will use a REST API in the next URL: <REST_URL>
    2. the REST API will get the user prompt in a parameter called: user_prompt
    3. the REST API support POST request only
    4. The REST API response will be in a parameter called: ai_reply
    5. The page title will be: “”” MTA Chat “””
    6. In the page, when I press Enter, I want it to press the SEND button
    7. Take the ai_reply and put it in the window

3. Test the HTML. It should look like this:



Task 10: Enhance the Lambda function to support conversation history

At this point, the Lambda function is not supporting conversation history, so open-ai will not reply to the prompt based on session context. As Lambda function is stateless, we will now add DynamoDB table to keep the conversation history and enhance the Lambda function to read and write to the table per needs. In addition, we will enhance the HTML page to pass to the Lambda API user_id.

1. Create a DynamoDB Table:
    - Table name: chat-history: A clear name representing stored chat messages.
    - Partition key name: user_id (String): Groups messages by user.
    - Sort key name: timestamp (Number): Ensures messages are ordered chronologically per user.
2. Enhance the Lambda function to read the last 6 messages from the database table and send it to open-ai API sorted by timestamp, then to save the last prompt and reply in the table. Remember also to get from the body the *user_id* value which is needed for the table:
    - *In addition to* 'user_prompt', *it will also get* 'user_id'.
    - *It will create a resource to* 'dynamodb' to table: 'chat-history'.
    - *It will read the last 6 messages from a DynamoDB table and send it to open-ai API sorted by timestamp using the conversation_history*
    - *List.*
    - *I will then save the last prompt and reply in the table. Remember also to get from the body the user_id value which is needed for the table.*
3. Choose **Test** tab, and use the next 4 prompts, one by one, to test your code:

```
{
  "body": {
    "user_prompt": "How much is 1+1?",
    "user_id": "user1"
  }
}
```

```
{
  "body": {
    "user_prompt": "How much is 2+2?",
    "user_id": "user1"
  }
}
```

```
{
  "body": {
```

```
    "user_prompt": "How much is 3+3?",
    "user_id": "user1"
  }
}
```

```
{
  "body": {
    "user_prompt": "How much is 4+4?",
    "user_id": "user1"
  }
}
```

```
{
  "body": {
    "user_prompt": "What did I just asked u?",
    "user_id": "user1"
  }
}
```

4. Verify the last reply you got:

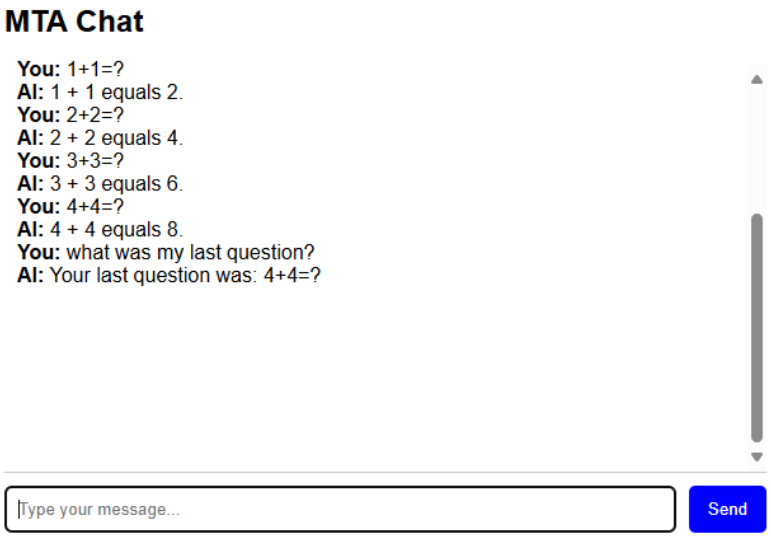
{
 "statusCode": 200,
 "headers": {
  "Access-Control-Allow-Origin": "*",
  "Access-Control-Allow-Methods": "OPTIONS,POST",
  "Access-Control-Allow-Headers": "Content-Type"
 },
 "body": "{\"ai_reply\": \"**The last question you asked me was \\\"4+4=?\\\"**\"}"
}

5. Verify the content of DynamoDB Table:

user_id *(String)*	timestamp *(Number)*	content	role
user6	1742153620204	1+1?	user
user6	1742153620205	1+1=2	assistant
user6	1742153631876	2+2?	user
user6	1742153631877	2+2=4	assistant
user6	1742153641787	3+3?	user
user6	1742153641788	3+3=6	assistant
user6	1742153650260	4+4?	user
user6	1742153650261	4+4=8	assistant
user6	1742153662156	what was my last question?	user
user6	1742153662157	Your last question was "4+4?"	assistant

6. Enhance the HTML page to send also the user_id. You can select one of the next 3 options:
   - Hard code the user_id value
   - Generate a random user_id
   - Ask the user for his name and use it as a user_id

7. Ask the 4 questions as noted above and verify the last one is correct:



8. Similarly, review your table:

☐	6a64d86d-5688-4b27-90dd-0412d0ac428a	1742154408633	1+1=?	user
☐	6a64d86d-5688-4b27-90dd-0412d0ac428a	1742154408634	1 + 1 equals 2.	assistant
☐	6a64d86d-5688-4b27-90dd-0412d0ac428a	1742154411613	2+2=?	user
☐	6a64d86d-5688-4b27-90dd-0412d0ac428a	1742154411614	2 + 2 equals 4.	assistant
☐	6a64d86d-5688-4b27-90dd-0412d0ac428a	1742154414790	3+3=?	user
☐	6a64d86d-5688-4b27-90dd-0412d0ac428a	1742154414791	3 + 3 equals 6.	assistant
☐	6a64d86d-5688-4b27-90dd-0412d0ac428a	1742154418607	4+4=?	user
☐	6a64d86d-5688-4b27-90dd-0412d0ac428a	1742154418608	4 + 4 equals 8.	assistant
☐	6a64d86d-5688-4b27-90dd-0412d0ac428a	1742154427795	what was my last question?	user
☐	6a64d86d-5688-4b27-90dd-0412d0ac428a	1742154427796	Your last question was: 4+4=?	assistant

Task 11: Deploy the HTML in AWS Amplify

Deploy your HTML in AWS Amplify as a static web page. Verify your link works.

Task 12: Add Chat history downloads as TXT file

In this task, you will extend the chat application with the ability to download the user's chat history. Your goal is to use appropriate AWS services to generate a **TXT** file containing the conversation and provide the user with a link to download it. All the TXT generated files, should be kept in AWS for future use as need. To complete this task, consider what needs to change on the **server** side, what must be updated on the client side, and how to redeploy the updated application to AWS Amplify.

Task 13: Add Chat history downloads as PDF file

In this task, you will extend the chat application with the ability to download the user's chat history. Your goal is to use appropriate AWS services to generate a **PDF** file containing the conversation and provide the user with a link to download it. To complete this task, consider what needs to change on the server side, what must be updated on the client side, and how to redeploy the updated application to AWS Amplify. **Hint: you will need to create/locate a lambda layer for the PDF creation.**

Task 14: Add a login page to your web site

In this task, you will extend the chat application with the ability to **make a login using user/password**. Make sure to pass the username from the login page to the chat page. To complete this task, consider what needs to change on the server side, what must be updated on the client side, and how to redeploy the updated application to AWS Amplify.

Activity completed

Congratulations! You have completed the activity.